

LivingBetterCC — Reporte de estado del proyecto

Fecha: 2025-12-27

1) Resumen ejecutivo

Este proyecto es una aplicación Django (5.1.x) con múltiples apps (authentication, classroom, dashboard, shop, etc.) y un sistema de formularios dinámicos (`authentication.formbuilder`).

Estado actual:

- El proyecto corre sin errores de `manage.py check`.
- Se migró completamente de CKEditor4 (`django-ckeditor`) a CKEditor5 (`django-ckeditor-5` / `django\_ckeditor\_5`).
- Se corrigió el endpoint de subida para CKEditor5 y el error `NoReverseMatch`.
- Se corrigió un problema de configuración donde Django estaba usando un URLConf equivocado.
- Se arregló la subida de imágenes en `formbuilder` y se implementó soporte para dos imágenes laterales distintas (izquierda/derecha).
- Se normalizó el detalle de "formularios completados" para que tenga el mismo encabezado visual que el render del formulario y se agregó botón "Imprimir".

2) Objetivos logrados (cambios principales)

2.1 CKEditor: migración total a CKEditor5

Problemas iniciales:

- Error `NoReverseMatch` por nombre de URL de upload mal configurado.
- Warning de seguridad de CKEditor4.

Acciones realizadas:

- Configuración: se ajustó `CK\_EDITOR\_5\_UPLOAD\_FILE\_VIEW\_NAME` para apuntar al nombre real de la vista de upload de `django\_ckeditor\_5`.
- Dependencias: se eliminó `django-ckeditor` (CKEditor4) y se dejó solo `django-ckeditor-5`.
- Modelos: campos rich text migrados a `CKEditor5Field` en apps donde aplicaba.
- Migraciones históricas: se limpiaron imports de `ckeditor` en migraciones antiguas reemplazándolos por `models.TextField()` para no depender del paquete removido.

Resultado:

- `manage.py check` sin warnings/errores.
- Los widgets CKEditor5 se inicializan correctamente cuando el HTML conserva la clase `django\_ckeditor\_5` y se incluye `form.media`.

2.2 Bootstrap vs CKEditor5 (toolbar no aparecía)

Causa:

- Algunos formularios sobre-escribían `widget.attrs['class'] = 'form-control'`, lo que removía la clase ``django_ckeditor_5``. El JS de CKEditor5 busca esa clase para inicializar.

Solución:

- Se cambió el patrón a “append class” (agregar sin borrar) en varios ``forms.py``.
- Se corrigieron ``__init__`` mal indentados dentro de ``Meta`` en ``classroom/courses/forms.py``.

2.3 URLConf incorrecto (404 usando otro proyecto)

Síntoma:

- Algunas rutas devolvían 404 y la página indicaba que se estaba usando un ``URLconf`` de otro módulo.

Solución:

- Se forzó ``DJANGO_SETTINGS_MODULE='livingbettercc.settings'`` en ``manage.py`` para evitar que variables de entorno apunten a otro proyecto.

2.4 Subida de imágenes en FormBuilder

Síntoma:

- Se “subía” una imagen en el CRUD de `FormDefinition` pero luego no se mostraba.

Causa:

- El template del CRUD no tenía ``enctype="multipart/form-data"``.
- La vista no pasaba ``request.FILES`` al ``ModelForm``.

Solución:

- Se agregó ``enctype="multipart/form-data"`` en la plantilla.
- ``form_create`/`form_update`` ahora usan ``FormDefinitionForm(request.POST, request.FILES, ...)``.

2.5 Dos imágenes laterales distintas (izquierda/derecha)

Objetivo:

- Permitir que el usuario suba dos imágenes diferentes y mostrarlas a la izquierda y derecha del encabezado del formulario.

Implementación:

- Se agregaron campos ``image_left`` e ``image_right`` al modelo ``FormDefinition``.
- Se mantuvo ``image`` como campo legacy para compatibilidad.
- Templates:

- ``render_form.html``: ahora muestra ``image_left`` / ``image_right``.

- ``completed_form_detail.html``: se alinea visualmente al mismo header (izq + logo + título + der).

Migración:

- Se generó y aplicó la migración ``formbuilder.0009_...image_left...image_right``.

2.6 Detalle de formulario completado (UI + imprimir)

Objetivo:

- Hacer que ``completed_form_detail.html`` se vea organizado como ``render_form.html``.
- Agregar botón ``Imprimir``.

Implementación:

- Se reestructuró el header para coincidir con el layout centrado.
- Se agregó un botón que ejecuta ``window.print()``.

3) Estado actual por módulo

authentication.formbuilder

- CRUD de formularios dinámicos funcional.
- Render público/privado según autenticación.
- Manejo de anexos (múltiples archivos) guardando URLs en JSON.
- FormDefinition soporta: ``description`` rich text (CKEditor5) y 3 campos de imagen (``image_left``, ``image_right``, ``image``).

classroom / dashboard / shop

- Modelos rich text migrados a CKEditor5.
- Forms ajustados para no romper clases del widget.

4) Riesgos / pendientes recomendados

- Consolidar el patrón ``_append_css_class`` en un helper común para evitar duplicación.
- Verificar que todas las páginas que usan CKEditor5 incluyen ``{{ form.media }}`` en el template (idealmente en base templates si aplica).
- Revisar que el entorno activo sea el correcto (hay múltiples venvs en el workspace).

5) Checklist de validación rápida

- ``manage.py check`` => OK
- Crear/editar FormDefinition y subir ``image_left`` y ``image_right`` => deben verse en el render.
- Abrir una página con CKEditor5 y confirmar toolbar visible.
- Ver detalle de formulario completado y usar botón ``Imprimir``.